

Human Activity Recognition Using Accelerometer and Gyroscope Data

Project Report

Hackathon 1 – Ask The Sensors

Prepared by:

A R Ashiq - 26AT63R06

Mohd Huzaifa Ansari - 26AT63R13

Shubhendra Pratap - 26AT63R09

Contents

1. Introduction
2. Objectives and Problem Statement
3. Dataset Description
4. Initial Dataset Analysis
5. Problems Identified
6. Proposed Solutions
7. Data Preprocessing and Sensor Synchronization
8. Windowing and Input Representation
9. Dataset Construction and Storage
10. Data Normalization
11. Machine Learning Methodology
12. Results and Discussion
13. Limitations and Future Work
14. Conclusion

1. Introduction

Human Activity Recognition (HAR) is the task of identifying a person's physical activity from sensor measurements. In this project, inertial measurements from an accelerometer and a gyroscope are used to distinguish among six physical activities: lying, sitting, walking, running, bicycling, and standing.

The project aims to develop a data-driven activity recognition pipeline that can learn temporal patterns directly from multi-axis inertial sensor signals. The main motivation is to move from raw, heterogeneous sensor recordings to a consistent machine-learning-ready representation while maintaining a fair evaluation on users that were not used during training.

2. Objectives and Problem Statement

The raw dataset is large and not directly suitable for supervised learning. Accelerometer and gyroscope measurements are stored separately, the two sensors do not initially share a common sampling timeline, label files use recording identifiers that differ from the internal sensor timestamps, and the activity classes are highly imbalanced. A complete preprocessing and learning pipeline is therefore required before model training.

- Create a common time base for accelerometer and gyroscope signals.
- Resample both sensors to a fixed sampling frequency of 25 Hz.
- Associate synchronized recordings with physical activity labels.
- Convert continuous recordings into fixed-size 5-second windows.
- Construct user-wise train, validation, and test splits to avoid user leakage.
- Normalize the six sensor channels using statistics computed only from the training set.

3. Dataset Description

The raw sensor data occupies approximately 31 GB. The dataset contains separate accelerometer and gyroscope recordings together with compressed label files. Because of the size of the raw data, the pipeline processes users and recordings incrementally rather than loading the complete dataset into memory.

After synchronization, the data is organized as:

sync/<USER_UUID>/<RECORDING_ID>/synchronized_25hz.csv

Column	Meaning
timestamp	Common sensor time axis
ax, ay, az	Three-axis accelerometer measurements
gx, gy, gz	Three-axis gyroscope measurements

The label data is organized as:

labels/<USER_UUID>.features_labels.csv.gz

The label files contain a timestamp/recording identifier and many activity or context columns.

For this project, the physical activity columns used as targets are

label:LYING_DOWN, label:SITTING, label:FIX_walking, label:FIX_running, label:BICYCLING, and label:OR_standing.

A value of 1 is treated as a positive label, 0 as a negative value, and NaN as unavailable/unannotated for that activity.

Class ID	Activity	Source label
0	Lying	label:LYING_DOWN
1	Sitting	label:SITTING
2	Walking	label:FIX_walking
3	Running	label:FIX_running
4	Bicycling	label:BICYCLING
5	Standing	label:OR_standing

4. Initial Dataset Analysis

The project uses a target sampling frequency of 25 Hz. This corresponds to a sample interval of 0.04 seconds. A fixed sampling frequency is important because the CNN expects samples with a consistent temporal spacing.

Two different notions of time appear in the data. Recording identifiers such as 1449601597 are used to map a sensor recording to its corresponding label entry, while sensor timestamps such as 440320.0856... form the internal time axis of the sensor samples. These should not be treated as the same quantity, and the preprocessing pipeline does not convert one into the other.

The accelerometer and gyroscope recordings for a given recording are not assumed to have perfectly identical timestamps. The preprocessing step therefore identifies their overlapping time interval and creates a shared 25 Hz timeline before interpolation.

Category	Count
Label users	58
Sensor users	57
Common users	55
Label-only users	3
Sensor-only users	2

Only the 55 common users can participate in the supervised-learning pipeline because both sensor data and labels are required.

5. Problems Identified

- Separate sensor files: accelerometer and gyroscope measurements are stored independently.
- Different sampling/timestamp behavior: the sensors do not directly form a single fixed-rate sequence.
- Large raw dataset: approximately 31 GB makes whole-dataset in-memory processing impractical.
- Recording-to-label association: recording identifiers and internal sensor timestamps serve different purposes.
- Missing or ambiguous activity labels: a recording may have no positive physical activity label or more than one positive physical activity label.

- Class imbalance: lying and sitting dominate the dataset, while running and bicycling have far fewer windows.
- User contribution imbalance: In the dataset description it was mentioned that the sampling rate of 40Hz for a burst time of 20 sec was used while recording the gyroscope and accelerometer readings which comes out to 800 records in 20 sec burst time, but for some users it was less than 800 records.

6. Proposed Solutions

Problem	Solution adopted
Separate accelerometer/gyroscope files	Pair corresponding recordings and create one synchronized file containing all six sensor channels.
Different time bases / rates	Find the common time interval and interpolate both sensors onto a 25 Hz timeline.
Very large dataset	Process one user/recording at a time and store fixed-size samples in chunked NPZ files.
Ambiguous recording labels	Keep recordings for the initial supervised dataset only when exactly one of the six target physical activities is positive.
Class imbalance	Use moderated class weights in the training loss instead of immediately deleting large amounts of common-class data.
Different sensor scales	Normalize each of the six channels with mean and standard deviation computed from the training data only.
Skewed dataset	Used class weights

7. Data Preprocessing and Sensor Synchronization

Each recording contains accelerometer and gyroscope measurements stored separately. The preprocessing code loads the timestamp and x, y, z channels from both sources for matching recordings.

The two sensors are first restricted to the time interval in which both have valid measurements. This avoids extrapolating one sensor beyond the period covered by the other.

A common timeline is created with a time step of 0.04 seconds. The timeline is therefore sampled at 25 Hz.

The original accelerometer and gyroscope samples are interpolated onto the common timeline. After this operation, each time point contains six aligned sensor values: ax, ay, az, gx, gy, and gz.

Each recording is saved as `synchronized_25hz.csv`. The resulting row structure is timestamp + six sensor channels. This becomes the source from which the fixed-length windows are generated for training models.

The recording identifier from the label data is matched to the recording folder name. The label identifier is therefore used for dataset association; it is not converted into the sensor timestamp used for interpolation. For the initial dataset, recordings are accepted only when exactly one target physical activity column has a value of 1.

8. Windowing and Input Representation

A CNN cannot conveniently train on recordings of arbitrary length. Fixed-size windows convert each continuous recording into a collection of equal-sized samples. A 5-second duration also provides enough temporal context to capture motion patterns while keeping the input manageable.

The selected window duration is 5 seconds. At 25 Hz, the number of samples in one window is $25 \times 5 = 125$ samples.

The baseline uses non-overlapping windows, so the stride is also 5 seconds. For a 20-second recording with exactly 500 samples, this creates four windows.

Each input sample has shape 125×6 , where 125 is the number of time samples and 6 is the number of sensor channels. The PyTorch CNN rearranges this to batch $\times$ channels $\times$ time, i.e. [batch, 6, 125].

The main improvement over the initial CNN approach was the use of LSTM layers to capture temporal patterns in the sensor sequence, while the additional statistical features provide information about the overall characteristics of the activity.

9. Dataset Construction and Storage

The processed samples are stored in compressed NPZ chunks rather than individual CSV files. A complete chunk contains 10,000 windows and stores X, y, users, and recordings. This reduces the file-count problem and allows the training pipeline to load only the required chunk.

The 55 common users are split at the user level: 43 users for training, 6 for validation, and 6 for test. This is intended to measure generalization to people whose sensor patterns were not seen during model fitting.

Split	Users	Windows	Chunks
Train	43	1,112,283	112
Validation	6	147,849	15
Test	6	89,363	9

A custom SensorDataset maps a global sample index to the corresponding NPZ chunk and local row. It caches the currently accessed chunk and returns X and y as PyTorch tensors.

The initial DataLoader uses batch size 64. Training batches are shuffled while validation batches preserve their order. Data loading is currently configured with num_workers=0 as a simple baseline, with performance optimization left for later if disk I/O becomes a bottleneck.

10. Data Normalization

Accelerometer and gyroscope channels have different numeric ranges and variability. Without normalization, larger-scale channels can have a disproportionate influence on optimization. Standardization gives the network a more balanced numerical input.

Channel	Mean	Std
ax	-0.00367690	1.41806539
ay	0.14116449	1.89913065
az	4.64917842	5.79612850
gx	0.00217559	0.27798988
gy	-0.00362624	0.30444976
gz	0.00570386	0.22924777

For every value x , normalization is performed as:

$$\mathbf{x_norm} = (\mathbf{x} - \mathbf{mean_train}) / \mathbf{std_train}$$

The statistics are calculated using the training split only and then reused for validation and test. This prevents information from the validation or test distributions from entering preprocessing.

A second giant normalized dataset is not created. Instead, the PyTorch Dataset normalizes each sample when it is loaded. A DataLoader test produced tensors of shape [64, 125, 6], confirming the expected sample representation.

11. Machine Learning Methodology

The first model is a one-dimensional convolutional neural network because the input is a temporal sequence. The convolution operates along the time dimension while the six sensor axes form the input channels.

The training set is strongly imbalanced. The initial inverse-frequency weights are too aggressive for the rarest class, especially running, so moderated square-root weights are used for the baseline training loss.

Class	Moderated weight
Lying	0.5588
Sitting	0.7243
Walking	1.8535
Running	8.1324
Bicycling	3.6152
Standing	1.4016

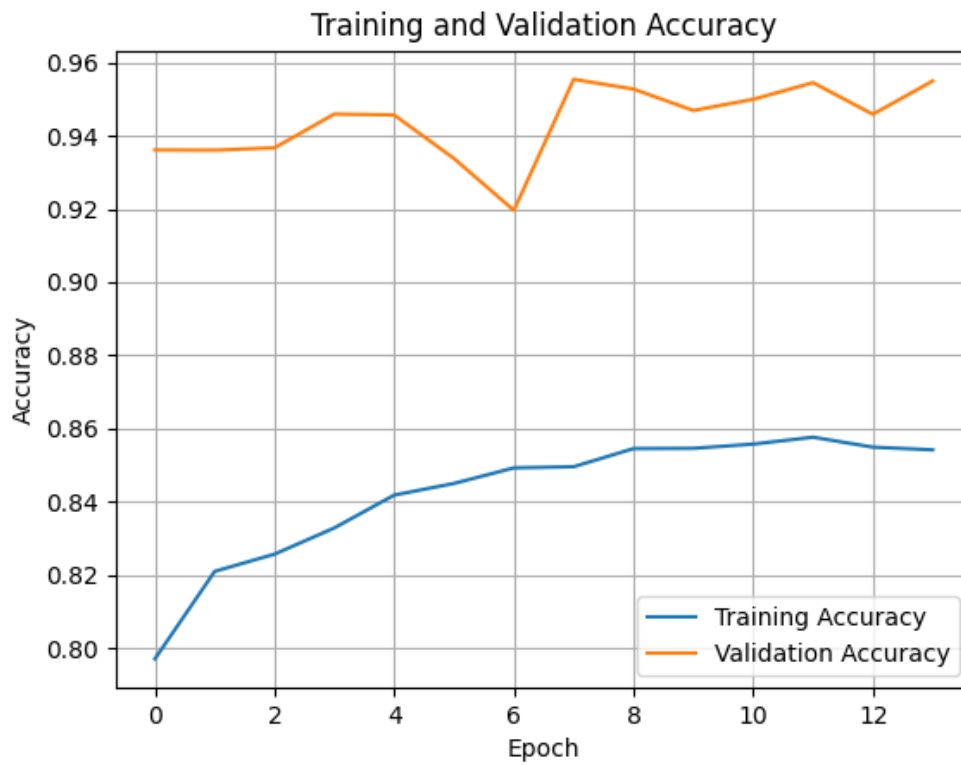
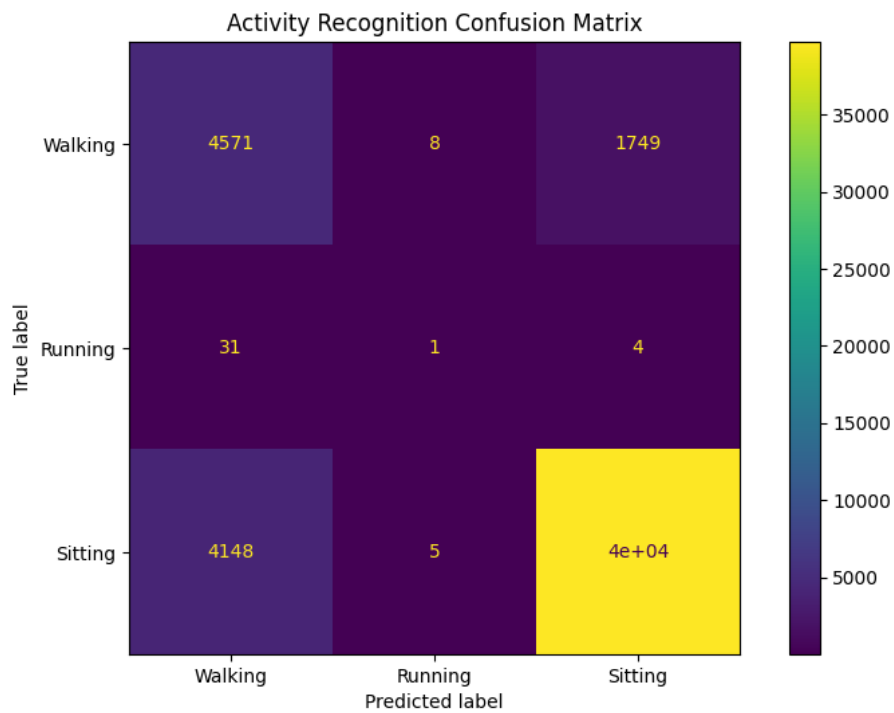
These weights affect only the training loss. They do not change the validation/test labels and do not mean that validation or test samples are artificially reweighted.

The planned baseline optimizer is Adam with learning rate 0.001.

Training is first intended to run for one epoch as an end-to-end pipeline check. Once the pipeline is confirmed, a longer run of approximately 10 epochs can be used for the first baseline experiment. The Apple Silicon MPS backend should be checked before training so that the available hardware acceleration can be used.

Because the dataset is highly imbalanced, model selection should prioritize validation macro-F1 rather than raw accuracy. Accuracy, macro precision, macro recall, macro-F1, per-class F1 scores, and the confusion matrix should all be reported. The test set should remain untouched during model development and should only be used for final evaluation.

12. Results on LSTM Training and Validation





API Endpoints and Query Engine Outputs

Activity Prediction Query API 1.0.0 OAS 3.1

/openapi.json

Query consolidated activity predictions generated by prediction.py.

System

GET /health Health

Predictions

GET /activities Activities

GET /predictions Predictions

GET /predictions/at Predictions At

Question Engine

POST /query Query Prediction

activity-tracker > http://127.0.0.1:8000/query Save Share

POST http://127.0.0.1:8000/query Send

Docs Params Authorization Headers 10 **Body** Scripts Settings Cook

raw JSON Schema Beautify

```
1 {"question": "how long was the user running?"}
```

Body 200 OK • 10 ms • 479 B • Save Response

JSON Preview Visualize Copy

```
2 "question": "how long was the user running?",
3 "answer": "You spent about 0.1 minutes running today, across 1 segment(s).",
4 "evidence": [
5   {
6     "activity": "RUNNING",
7     "start_timestamp": 36003.30648625,
8     "end_timestamp": 36007.26648625,
9     "start_ms": 36003306.486250006,
10    "end_ms": 36007266.486250006,
11    "start_time": "36003.3",
12    "end_time": "36007.3",
13    "duration_s": 3.9599999999999127
14  }
15 ]
16 }
```

activity-tracker > http://127.0.0.1:8000/query

POST http://127.0.0.1:8000/query Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {"question": "how long was the user sitting between 600 and 700?"}
```

Body 200 OK • 14 ms • 695 B • Save Response

{ JSON Preview Visualize

```
1 {
2   "question": "how long was the user sitting between 600 and 700?",
3   "answer": "You spent about 1.6 minutes sitting during this time frame, across 2
4     segment(s).",
5   "evidence": [
6     {
7       "activity": "SITTING",
8       "start_timestamp": 600.0,
9       "end_timestamp": 614.4219915000001,
10      "start_ms": 600000.0,
11      "end_ms": 614421.9915000001,
12      "start_time": "600",
13      "end_time": "614.422",
14      "duration_s": 14.421991500000104
15    }
16  ]
17 }
```

activity-tracker > <http://127.0.0.1:8000/query> Save Share

GET <http://127.0.0.1:8000/predictions> Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

none

This request does not have a body

Body 200 OK • 11 ms • 74.34 KB Save Response

{ } JSON Preview Visualize

```
1 [
2   {
3     "activity": "Walking",
4     "start_timestamp": 556.6282185,
5     "end_timestamp": 564.5882185
6   },
7   {
8     "activity": "Sitting",
9     "start_timestamp": 564.6282185,
10    "end_timestamp": 614.4219915000001
11  },
12 ]
```

activity-tracker > <http://127.0.0.1:8000/query> Save Share

GET <http://127.0.0.1:8000/predictions/at?timestamp=600> Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

none

This request does not have a body

Body 200 OK • 17 ms • 213 B Save Response

{ } JSON Preview Visualize

```
1 [
2   {
3     "activity": "Sitting",
4     "start_timestamp": 564.6282185,
5     "end_timestamp": 614.4219915000001
6   }
7 ]
```

At the current stage, the preprocessing pipeline, dataset construction, normalization, and baseline model design are prepared, but final CNN training metrics have not yet been generated. Therefore, no final accuracy, precision, recall, F1-score, or confusion matrix is reported here.

This section should be filled after the first training run with epoch-wise training loss, training accuracy, and macro-F1.

This section should contain validation loss and macro-F1 for each epoch, together with the best checkpoint selected according to validation macro-F1.

The final test metrics should be calculated only after model selection is complete. Test performance should be presented separately from development results.

A six-class confusion matrix should be included to show which activities are most frequently confused, for example sitting versus standing or walking versus running, once actual predictions are available.

Particular attention should be given to running and bicycling because they are much rarer than lying and sitting. A model with high overall accuracy but poor minority-class recall would not be considered satisfactory for the project objective.

13. Limitations and Future Work

- The baseline uses non-overlapping windows; overlapping windows can be evaluated later to increase temporal coverage.
- The initial labeling rule discards recordings with no positive or multiple simultaneous target activity labels; a more nuanced labeling policy may recover additional data.
- Class imbalance remains substantial even after class-weighted loss and may require additional strategies such as careful resampling or augmentation.
- User contribution is uneven, so future experiments may consider ways to reduce the influence of users with extremely large numbers of windows.
- The first model is intentionally simple. Deeper CNNs, residual architectures, attention-based models, or CNN-RNN combinations can be compared later.
- Hyperparameters such as learning rate, batch size, window length, and model depth can be tuned after the baseline is validated.
- Inference-time efficiency and deployment on embedded or edge hardware can be investigated as a later CPS-oriented objective.

14. Conclusion

The project has established a complete foundation for human activity recognition from accelerometer and gyroscope data. The raw multi-user dataset has been transformed from separate large-scale sensor recordings into synchronized 25 Hz six-channel sequences, associated with physical activity labels, segmented into 5-second windows, stored in scalable chunks, split by user, normalized using training-only statistics, and prepared for a class-weighted 1D CNN. The next critical stage is to execute and evaluate the baseline training run and then use the measured results to guide improvements.

So far, the LSTM model has been trained and evaluated using data from 15 users. After evaluating the initial LSTM results, we are now moving towards a 1D Convolutional Neural Network (CNN1D) approach. CNN1D is suitable for this project because accelerometer and gyroscope data are time-series signals containing

local temporal patterns that can be effectively learned using convolutional filters. It also provides a comparatively efficient approach for processing sensor data.

The data processing and preprocessing stage for the complete dataset of 60 users has now been completed. The processed dataset is currently being used to train the CNN1D model on data from all 60 users. This will allow us to evaluate the model on a larger and more diverse dataset and compare its performance with the previously trained LSTM model.

AI Usage:

This project utilized AI-assisted tools (ChatGPT, Github Copilot) as a supplementary aid during development. These tools were primarily used for generating initial code templates, debugging assistance, and clarifying implementation details.

All core aspects of the project were designed and implemented by us, including:

- Selection and design of the model architecture
- Data preprocessing pipeline (windowing, normalization, and feature engineering)
- Choice of evaluation metrics and training strategy
- Hyperparameter tuning and performance optimization
- Integration, testing, and validation of the complete system

AI-generated outputs were not used directly without verification. Each suggestion was critically evaluated, modified where necessary, and validated through experimentation. Several iterations of the model and pipeline were explored before arriving at the final implementation.

Thus, AI served as a productivity tool, while the overall system design, decision-making, and final implementation reflect our independent work.